

透過 MCP 和 Cloud Run 部署 Enterprise Governance-Aware 代理

來源：<https://codelabs.developers.google.com/governance-context-part2?hl=zh-tw>

步驟數：7

在本系列的第 2 部分中，您會將本機 AI 原型升級為正式環境級別的企業應用程式。您會在 Cloud Run 部署 Model Context Protocol (MCP) 伺服器，做為集中式資料控制平面，並使用 Google 的 Agent Development Kit (ADK) 建構、協調及部署安全且具治理意識的代理，為使用者提供內建的網頁介面。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [透過 MCP 擴大規模：建構資料控制層](#)
4. [使用 ADK 建構代理後端](#)
5. [測試企業代理](#)
6. [清除所用資源](#)
7. [恭喜！](#)

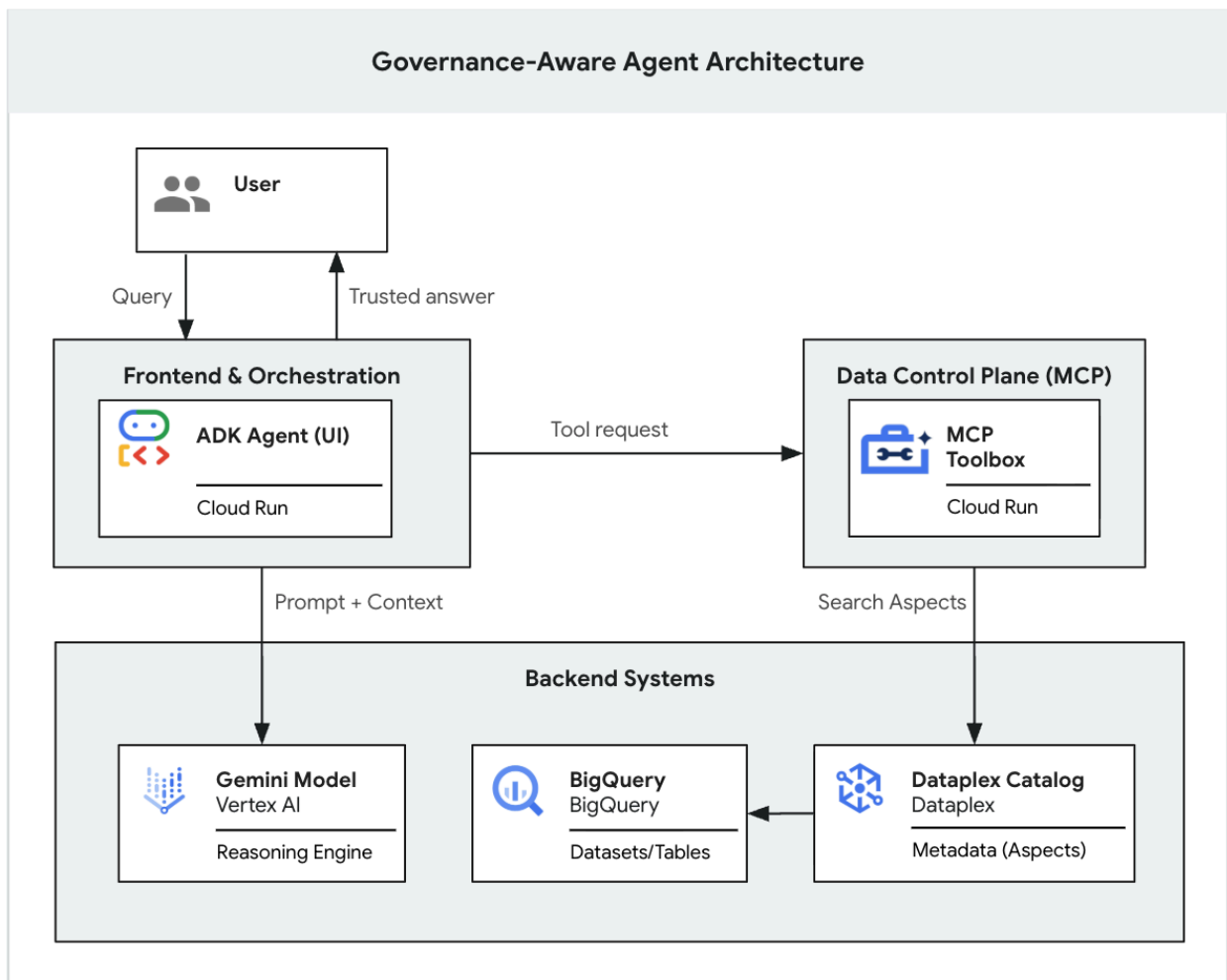
1. 簡介

本程式碼研究室是兩部分系列的第一部分，探討如何建構具備治理意識的生成式 AI 代理程式。

(您可以參閱本系列的第一部分，瞭解如何將 Dataplex 構面套用至 BigQuery 資料表，並透過 Gemini CLI 在本機測試規則，建立資料基礎。👉 [閱讀第 1 部分](#))

不過，在本機 CLI 中測試只是第一步。如要向全公司推出這項功能，您需要集中式安全防護、標準化 AI 工具連線，以及適當的應用程式框架，以便自動化調度管理代理的邏輯，並提供熟悉的即時通訊介面。

在第二部分中，您將解決這些挑戰，並擴大規模以供正式環境使用。您會將控管規則部署至託管於 [Cloud Run](#) 的中央 [MCP 伺服器](#)。接著，您將使用 Google 的 [Agent Development Kit \(ADK\)](#) 建構實際的代理應用程式，並將其連線至 MCP 工具，完成專業的網頁 UI。



必要條件

- 已啟用計費功能的 Google Cloud 專案。
- 對 Cloud Run、IAM 服務帳戶和 Python 有基本瞭解。
- 在第 1 部分中建立的 **BigQuery** 資料集和 **Dataplex** 構面。(如果已刪除，請別擔心，我們會在下方提供快速建立這些檔案的指令碼！)

課程內容

- 如何使用 Model Context Protocol (MCP)，將 AI 代理與 Google Cloud 資料的互動方式標準化。

- 如何將安全的 MCP 伺服器部署至 Cloud Run。
- 如何使用 Agent Development Kit (ADK) 建構 AI 代理，並將其連線至 MCP 後端。
- 如何執行 ADK 內建的開發人員 UI，與受控代理互動。

軟硬體需求

- 存取 Google Cloud Shell

重要概念

- **Model Context Protocol (MCP)**：不妨將 MCP 視為 AI 代理程式的「通用 USB-C 傳輸線」。不必為每個 AI 模型編寫自訂 API 整合程式碼，MCP 提供標準做法，讓 AI 安全地連結至企業資料工具 (例如 Dataplex 和 BigQuery)。
 - **Agent Development Kit (ADK)**：Google 開發的彈性開放原始碼架構，旨在簡化 AI 代理的端對端開發作業。這項工具將軟體工程原理套用至代理程式建立作業，讓您能自動化調度管理複雜工具、管理狀態，以及輕鬆啟動內建的開發人員 UI，進行測試和部署。
-

步驟 2 · 約 0 分鐘

2. 設定和需求

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell interface. The browser tab is titled 'qwiklabs-gcp-b3de7b9575cb8d19 x'. The terminal window has a dark background and displays the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by a prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

初始化環境

開啟 Cloud Shell 並設定專案變數，確保所有指令都以正確的基礎架構為目標。

```
export PROJECT_ID=$(gcloud config get-value project)
gcloud config set project $PROJECT_ID
export REGION="us-central1"
```

查核點：要繼續還是重建？

由於這是第 2 部分，代理程式需要第 1 部分的受管理資料才能運作。請選擇適合自己的方式：

路徑 A：我剛完成第 1 部分，資源仍在執行中。

太好了！前往工作目錄，即可繼續操作。

```
cd ~/devrel-demos/data-analytics/governance-context
```

路徑 B：我略過第 1 部分，或是刪除資源 (已清除)。

沒問題！我們在下方提供「快速通道」指令區塊。這會自動重建 BigQuery 資料湖泊，並套用 Dataplex 管理中繼資料，與第 1 部分的做法完全相同。

```
# 1. Clone the repo and navigate to the working directory
git clone --depth 1 --filter=blob:none --sparse
https://github.com/GoogleCloudPlatform/devrel-demos.git
cd devrel-demos
git sparse-checkout set data-analytics/governance-context
cd data-analytics/governance-context

# 2. Rebuild the messy data lake with Terraform
cd terraform
terraform init
terraform apply -var="project_id=${PROJECT_ID}" -var="region=${REGION}" -auto-approve

# 3. Generate and apply Dataplex Aspects (Governance rules)
cd ..
chmod +x ./generate_payloads.sh ./apply_governance.sh
./generate_payloads.sh
./apply_governance.sh
```

3. 透過 MCP 擴大規模：建構資料控制層

到目前為止，您已使用 Gemini CLI 成功測試控管邏輯。這非常適合快速製作原型，但會使用您的個人使用者憑證在本機執行。

在實際的企業環境中，您需要集中式資料控制層。為建構這項功能，我們將使用 Google 的官方開放原始碼專案「[資料庫專用的生成式 AI 工具箱](#)」。這個工具箱提供預先建構的 MCP 伺服器，專門用於將 AI 代理安全地連線至 Google Cloud 資料庫和中繼資料服務 (例如 Dataplex)。

將這個工具箱部署為 Cloud Run 上的 MCP 伺服器，可達成下列目標：

1. **集中式身分**：代理程式會以受限的服務帳戶執行，而非您的個人使用者帳戶。
2. **標準化**：任何用戶端 (ADK、Gemini、自訂應用程式) 都能使用標準 MCP 通訊協定「外掛」至這個伺服器。
3. **受控範圍 (最低權限)**：我們不會授予 LLM 無限制的 BigQuery 存取權，我們強制系統先透過 Dataplex 中繼資料目錄導覽。

設定工具定義 (`tools.yaml`)

生成式 AI 工具箱需要宣告式設定檔 `tools.yaml`。這個檔案會定義 `sources` (連線位置) 和 `tools` (AI 可執行的動作)。

1. 前往伺服器目錄，並將專案 ID 插入設定檔：

```
cd ~/devrel-demos/data-analytics/governance-context/mcp_server
envsubst < tools.yaml > tools.tmp && mv tools.tmp tools.yaml
cat tools.yaml
```

畫面應與下列程式碼片段相同。確認專案欄位現在與實際的 **Google Cloud 專案 ID** 相符。

```
sources:
  dataplex:
    kind: dataplex
    project: YOUR-PROJECT-ID

tools:
  search_entries:
    kind: dataplex-search-entries
    source: dataplex
    description: Search for entries in Dataplex Catalog.

  lookup_entry:
    kind: dataplex-lookup-entry
    source: dataplex
    description: Retrieve a specific entry from Dataplex Catalog.

  search_aspect_types:
    kind: dataplex-search-aspect-types
    source: dataplex
    description: Find aspect types relevant to a query.

toolsets:
  dataplex-toolset:
    - search_entries
    - lookup_entry
    - search_aspect_types
```

🔍 深入瞭解：治理推論迴圈

請仔細查看 YAML 中的工具部分。我們明確公開了三項特定的 Dataplex 工具。這並非隨機行為，而是形成嚴格的推論迴圈，避免出現幻覺：

- **dataplex-search-aspect-types (瞭解規則)**：使用者要求「董事會核准的資料」時，LLM 不知道這是什麼意思。這項工具會用來搜尋 Dataplex 切面類型。系統發現「Board Approved」對應的層級為 GOLD_CRITICAL，且與名為 official-data-product-spec 的層面相符。
- **dataplex-search-entries (尋找候選項目)**：LLM 現在知道要尋找什麼，因此會使用這項工具搜尋 Dataplex Catalog (例如搜尋含有「收益」關鍵字的資料表)。並取得相符資源的清單。
- **dataplex-lookup-entry (驗證事實)**：在撰寫任何 SQL 之前，LLM 會使用這項工具檢查特定資料表的確切中繼資料。如果資料表未附加 GOLD_CRITICAL 方面，LLM 會拒絕並繼續搜尋。

定義這三項工具後，我們就能強制 AI 進入「唯讀」和「治理優先」模式。

保護設定 (Secret Manager)

在企業架構中，您絕不應直接將設定檔烘焙到容器映像檔中。我們會將 `tools.yaml` 安全地儲存在 [Google Cloud Secret Manager](#) 中。


```
gcloud services enable secretmanager.googleapis.com
gcloud secrets create dataplex-tools-config --data-file=tools.yaml
```

實作最小權限原則 (IAM)

接著，我們將為 GenAI Toolbox MCP 伺服器建立專用服務帳戶。這個身分僅具備讀取 Dataplex 目錄和存取 BigQuery 資料所需的確切權限。

```
export MCP_SA=mcp-sa
gcloud iam service-accounts create ${MCP_SA} \
  --display-name="Service Account for Dataplex MCP"
export MCP_SERVICE_ACCOUNT="${MCP_SA}@${PROJECT_ID}.iam.gserviceaccount.com"

# Allow the server to read its own config from Secret Manager
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$MCP_SERVICE_ACCOUNT" \
  --role="roles/secretmanager.secretAccessor"

# Allow the server to read Dataplex Metadata and BigQuery Data
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$MCP_SERVICE_ACCOUNT" \
  --role="roles/dataplex.catalogViewer"
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$MCP_SERVICE_ACCOUNT" \
  --role="roles/bigquery.dataViewer"
```

將 MCP 伺服器部署至 Cloud Run

現在，我們要部署生成式 AI 工具箱。我們使用 Google 預先建構的容器映像檔 (`database-toolbox/toolbox`)，並在執行階段從 Secret Manager (`--set-secrets`) 掛接設定。

```
export IMAGE=us-central1-docker.pkg.dev/database-toolbox/toolbox/toolbox:latest

gcloud run deploy governance-mcp \
  --image=$IMAGE \
  --service-account $MCP_SERVICE_ACCOUNT \
  --region=$REGION \
  --no-allow-unauthenticated \
  --set-secrets="/app/tools.yaml=dataplex-tools-config:latest" \
  --args="--tools-file=/app/tools.yaml","--address=0.0.0.0","--port=8080"
```

您現在已建立受控 API！生成式 AI 前端會連線至這個 Cloud Run 網址，而非直接存取資料庫。代理程式只能看到工具箱允許的內容。

4. 使用 ADK 建構代理後端

您已建立在 Cloud Run 上執行的安全受控資料控制平面 (MCP)。現在 AI 代理需要框架來協調其邏輯，例如處理使用者輸入內容、決定何時呼叫 MCP 伺服器，以及格式化輸出內容。

我們將使用 Google 的 Agent Development Kit (ADK)，而非從頭開始撰寫所有樣板程式碼。ADK 是以程式碼為優先的框架，可自動將代理程式邏輯包裝成 FastAPI 後端。此外，這項工具內建開發人員 UI，讓您不必先建構自訂前端，就能立即查看代理程式的推理過程和工具呼叫。

檢查代理程式邏輯 (agent.py)

設定基礎架構前，先來看看這個應用程式的核心。

前往該目錄，並輸出 agent.py 的內容。這個檔案是 ADK 部署作業的「大腦」。

```
cd ~/devrel-demos/data-analytics/governance-context/mcp_server
cat agent.py
```

查看程式碼結構。這項工具可執行三項重要功能，且樣板最少：

- MCPToolset 整合：**ADK 會使用 `MCPToolset(server_url=mcp_url)`，而非編寫自訂 HTTP 用戶端與 Dataplex 工具互動。這項功能會從已部署的 MCP 伺服器動態擷取 `tools.yaml` 定義，並將其轉換為 LLM 的原生函式呼叫。
- 系統指令：**`instructions` 參數包含嚴格的控管規則 (與 CLI `GEMINI.md` 中使用的邏輯相同)，明確指示模型執行第 1 階段 (中繼資料查詢) 到第 2 階段 (資料查詢) 的推論迴圈。
- 代理自動調度管理：**`Agent(...)` 類別會將 Gemini 模型、系統提示和 MCP 工具繫結在一起。部署時，ADK 會自動將這個物件轉換為可擴充的 FastAPI 端點。

職責劃分：設定前端身分

如要安全地執行這段程式碼，我們必須告知代理程式 MCP 伺服器的位置。我們會動態建構網址，並儲存至 `.env` 檔案，ADK 會在執行階段讀取該檔案。

我們也會為這個面向使用者的應用程式建立個別身分 (`dataplex-agent-sa`)。職責分離可確保前端代理程式的權限與後端控管伺服器不同。

執行下列指令，設定環境和身分識別：

```
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")
export MCP_SERVER_URL=https://governance-mcp-${PROJECT_NUMBER}.${REGION}.run.app/mcp

export AGENT_SA=dataplex-agent-sa
export AGENT_SERVICE_ACCOUNT="${AGENT_SA}@${PROJECT_ID}.iam.gserviceaccount.com"

gcloud iam service-accounts create ${AGENT_SA} \
  --display-name="Service Account for Dataplex Agent "
```

設定執行階段變數

ADK 架構會根據環境變數瞭解背景脈絡。我們需要明確設定專案 ID、區域，並啟用 Vertex AI 用量。我們將這些內容附加至同一個 `.env` 檔案。

```
echo MCP_SERVER_URL=$MCP_SERVER_URL > .env
echo GOOGLE_GENAI_USE_VERTEXAI=1 >> .env
echo GOOGLE_CLOUD_PROJECT=$PROJECT_ID >> .env
echo GOOGLE_CLOUD_LOCATION=$REGION >> .env
```

授予權限

即使代理程式將控管檢查委派給 MCP 伺服器，仍需要基本權限才能運作。我們授予的角色有兩種：

1. **Vertex AI 使用者**：叫用 Gemini 模型生成自然語言回覆。
2. **Cloud Run 叫用者**：安全地呼叫 MCP 伺服器 API。但不會取得 BigQuery 或 Dataplex 的直接存取權！

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$AGENT_SERVICE_ACCOUNT" \
  --role="roles/aiplatform.user"

gcloud run services add-iam-policy-binding governance-mcp \
  --region=$REGION \
  --member="serviceAccount:$AGENT_SERVICE_ACCOUNT" \
  --role="roles/run.invoker"
```

部署至 Cloud Run

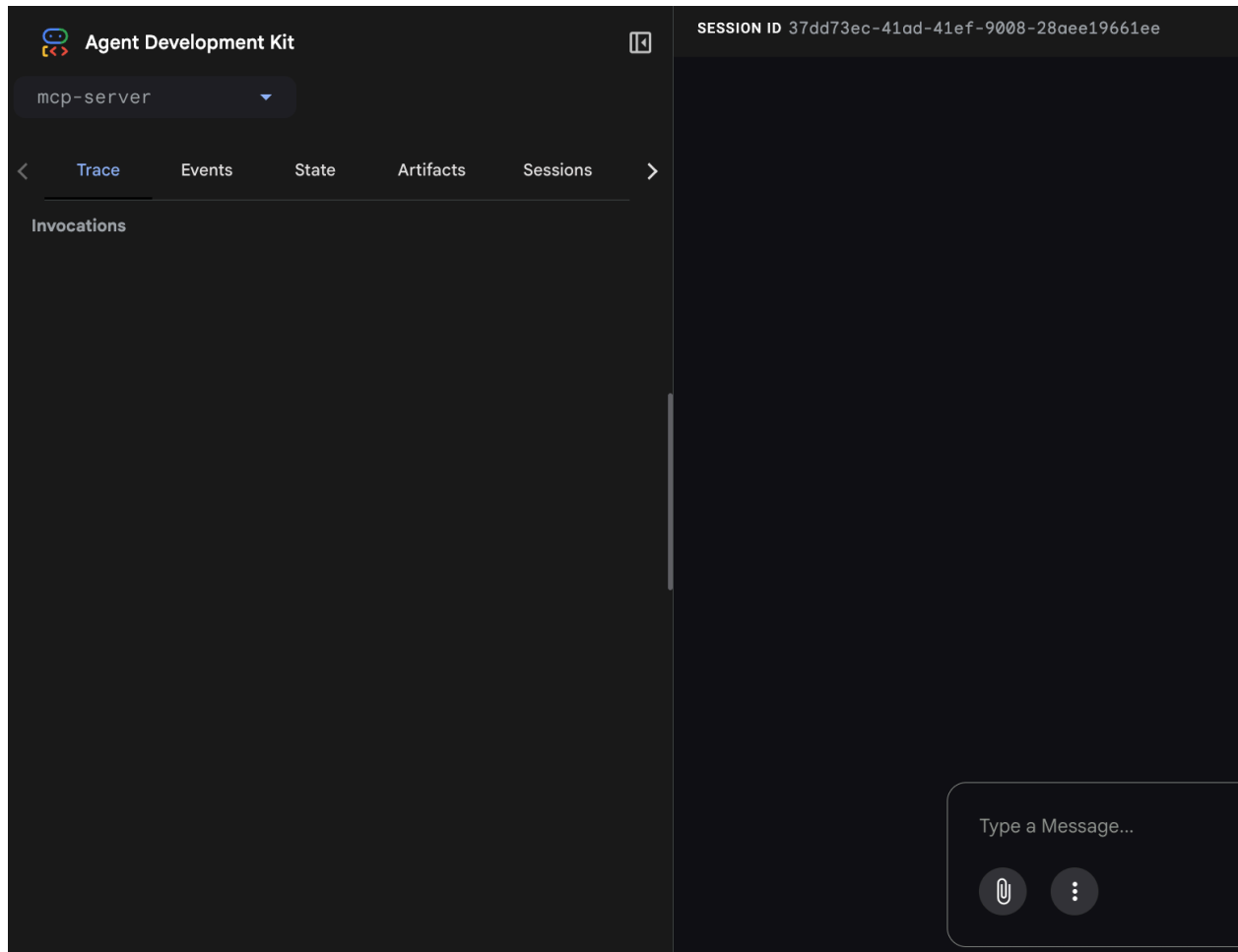
最後，我們將完整堆疊部署至 Cloud Run。

我們使用 `uvx` 執行 ADK 工具，不必手動安裝依附元件。下列指令會封裝 `agent.py` 邏輯、建構容器映像檔、插入服務帳戶，並啟動 FastAPI 伺服器。加入 `--with_ui` 標記後，系統也會一併封裝 ADK Web Playground，方便您進行偵錯。

這個指令會建構並部署容器。這項作業會在 1 至 3 分鐘內完成。

```
uvx --from google-adk \
  adk deploy cloud_run \
    --project=$PROJECT_ID \
    --region=$REGION \
    --service_name=dataplex-agent \
    --with_ui \
    . \
    -- \
    --service-account=$AGENT_SERVICE_ACCOUNT \
    --allow-unauthenticated
```

這項指令完成後，會輸出「服務網址」（e.g., `https://dataplex-agent-xyz.run.app`）。按一下該連結，即可開啟完全受控的生成式 AI 即時通訊介面。



端對端架構流程

您現在已完成系統。使用者與 ADK UI 互動時，會發生下列情況：

1. 使用者在 **ADK 代理 (開發人員使用者介面)** 中提交提示。
 2. **ADK 代理 (agent.py)** 會處理輸入內容，並呼叫 **Gemini** 模型。
 3. **Gemini** 判斷需要背景資訊，並要求 **MCP 伺服器**執行 Dataplex 工具。
 4. **MCP 伺服器**會強制執行 **Dataplex 管理規則**，並傳回中繼資料。
 5. **Gemini** 會根據中繼資料綜合判斷可信答案，並傳回給使用者。
-

5. 測試企業代理

現在代理已上線，讓我們重新測試先前透過 CLI 測試的管理情境。邏輯保持不變，但您現在是與已部署的 ADK Web Playground 互動，該平台會將內部狀態和工具執行作業視覺化。

1. 協調：ADK 代理 (在 Cloud Run 上執行) 會收到您的文字。
2. 工具轉送：Gemini 判斷您的問題需要資料背景資訊，並將要求轉送至 **MCP 伺服器**。
3. 控管檢查：MCP 伺服器 (在個別的 Cloud Run 執行個體上執行) 會查詢 Dataplex 的特定 Aspect 類型。
4. 綜合分析：系統會將相關中繼資料傳回 Gemini，生成最終答案。

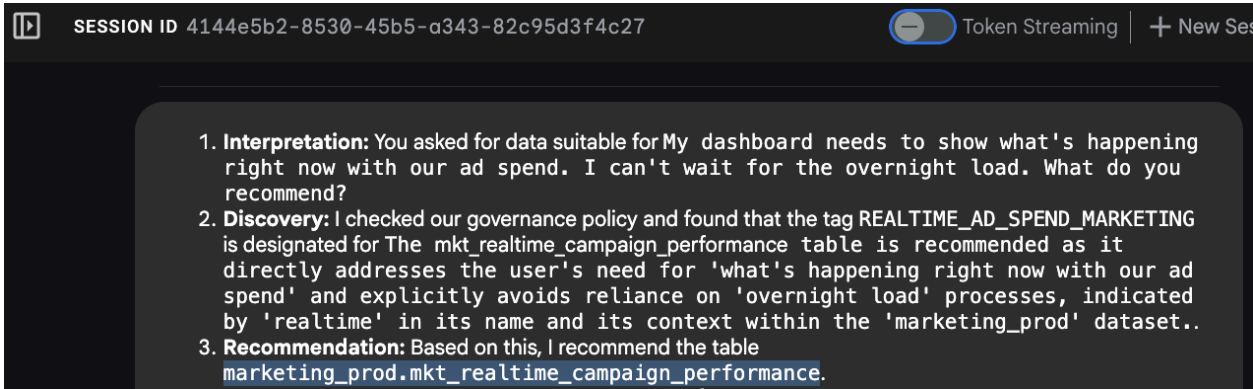
驗證控管邏輯

在瀏覽器中開啟上一步產生的「服務網址」 (e.g., <https://dataplex-agent-xyz.run.app>)。貼上下列提示詞：

"My dashboard needs to show what's happening right now with our ad spend. I can't wait for the overnight load. What do you recommend?"

在開發人員 UI 中觀察 Agent 的推論過程：

1. 意圖辨識：代理程式會剖析「現在」和「等不及過夜」。
2. 中繼資料查詢：呼叫 MCP 工具 `search_aspect_types`。系統會尋找「`update_frequency` 方面」設為「即時」或「串流」，而非「每日」或「每月」的資料資產。
3. 選取：系統會判斷資料表 `mkt_realtime_campaign_performance` 符合這些條件，而 `fin_monthly_closing_internal` (雖然品質很高) 對您的要求來說速度太慢。
4. 回覆：服務專員建議使用即時意見回饋表。



重要性：

如果沒有這類管理中繼資料，LLM 可能會因為 `fin_monthly_closing_internal` 資料表有名為「ad_spend」的資料欄，就建議使用該資料表，而忽略資料已過時 24 小時的事實。中繼資料內容可避免業務錯誤。

您也可以測試「董事會會議」提示詞，瞭解代理程式如何根據資料產品層級面向，透視至不同資料表：

"We are preparing the deck for an internal Board of Directors meeting next week. I need the numbers to be absolutely finalized, trustworthy, and kept strictly confidential. Which table is safe to use?"

步驟 6 · 約 0 分鐘

6. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作，刪除第 1 部分和第 2 部分建立的所有基礎架構。

刪除 Datalake (Terraform)

使用 Terraform 拆除 BigQuery 資料表、資料集和 Dataplex Aspect 定義。

```
cd ~/devrel-demos/data-analytics/governance-context/terraform
terraform destroy -var="project_id=${PROJECT_ID}" -var="region=${REGION}" -auto-approve
```

刪除 Cloud Run 服務

移除運算資源，停止收取執行中容器的費用。

```
gcloud run services delete governance-mcp --region=$REGION --quiet
gcloud run services delete dataplex-agent --region=$REGION --quiet
```

清理建構作業構件和暫存儲存空間

使用 `uvx` 部署 ADK 代理程式時，系統會自動建構容器映像檔，並將原始碼上傳至暫時的 Cloud Storage 值區。即使刪除 Cloud Run 服務，這些構件仍會保留，並持續產生儲存費用。

移除 Artifact Registry 存放區和 Cloud Storage 暫存 bucket：

```
# Delete the repository used for the agent build
gcloud artifacts repositories delete cloud-run-source-deploy \
  --location=$REGION \
  --quiet

# Delete the staging bucket created by Cloud Run source deploy
gcloud storage rm --recursive gs://run-sources-${PROJECT_ID}-${REGION}
```

刪除身分、權限和密鑰

請先移除 IAM 政策繫結，避免「墓碑」項目 (孤立記錄) 留在專案的 IAM 頁面。接著，刪除服務帳戶和設定密鑰。

```
# Remove IAM roles granted to the MCP Service Account
gcloud projects remove-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$MCP_SERVICE_ACCOUNT" \
  --role="roles/secretmanager.secretAccessor" --quiet
gcloud projects remove-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$MCP_SERVICE_ACCOUNT" \
  --role="roles/dataplex.catalogViewer" --quiet
gcloud projects remove-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$MCP_SERVICE_ACCOUNT" \
  --role="roles/bigquery.dataViewer" --quiet

# Remove IAM roles granted to the Agent Service Account
gcloud projects remove-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$AGENT_SERVICE_ACCOUNT" \
  --role="roles/aiplatform.user" --quiet

# Delete the Service Accounts
gcloud iam service-accounts delete $MCP_SERVICE_ACCOUNT --quiet
gcloud iam service-accounts delete $AGENT_SERVICE_ACCOUNT --quiet

# Delete the Secret Manager entry
gcloud secrets delete dataplex-tools-config --quiet
```

移除本機設定

最後，清理 Cloud Shell 中的本機設定檔和環境變數。

```
# Uninstall the Gemini CLI extension (installed in Part 1)
gemini extensions uninstall dataplex

# Remove local repository files and unset variables
cd ~
rm -rf ~/devrel-demos
unset MCP_SERVER_URL
unset MCP_SERVICE_ACCOUNT
unset AGENT_SERVICE_ACCOUNT
```

7. 恭喜！

您已成功部署端對端、具備管理意識的生成式 AI 代理程式。

在本程式碼研究室的兩部分中，您已超越簡單的提示工程，實作強大的正式環境就緒架構。將資料治理視為 GenAI 的先決條件，建立系統化方法，防止模型擷取未經認證或捏造的資料。

重點回顧

- **透過中繼資料實現確定性 AI**：您使用 GenAI Toolbox for Databases 強制執行嚴格的推論迴圈，而非依賴 LLM 根據欄名猜測正確的資料表。您只明確公開三項 Dataplex 工具 (`search_aspect_types`、`search_entries`、`lookup_entry`)，因此模型必須先驗證資料認證，才能綜合生成答案。
- **分離式架構 (MCP)**：在 Cloud Run 上部署 Model Context Protocol (MCP) 伺服器後，您已將資料治理規則抽象化為集中式標準化 API。前端代理程式不需要包含資料庫邏輯，只需要透過 MCP 標準通訊即可。也就是說，您可以在同一個受控後端中，插入任何未來的 AI 模型或用戶端。
- **職責分離**：您已隔離 IAM 身分，並套用最小權限原則。面向使用者的 ADK 代理程式只能叫用模型和進行 API 路由，後端的 MCP 伺服器則會安全地處理 Dataplex 目錄查詢和 BigQuery 資料擷取作業。
- **以程式碼為優先的代理協調**：您使用 Google Agent Development Kit (ADK)，將 Python 代理邏輯立即封裝到可擴充的 FastAPI 後端，並利用內建的開發人員 UI，將代理的內部工具執行作業視覺化及偵錯。

後續步驟

- [Dataplex 基礎治理 Codelab](#)：先掌握 Dataplex 的資料治理基礎知識，再新增 AI 層。
 - [Dataplex 工具說明文件](#)：探索本實驗室使用的預先建構 Dataplex 工具和擴充功能的官方說明文件。
 - [開始使用 Gemini CLI 擴充功能](#)：瞭解如何建構自訂擴充功能，進一步提升生成式 AI 代理程式的功能。
 - [深入瞭解 MCP](#)：請參閱官方 MCP 規格，瞭解如何為內部企業 API 建構自訂伺服器。
-